

RESUMEN EJECUTIVO DEL PROYECTO

NOMBRE	SAMUEL RICARDO CUBILLOS FUENTES
CEDULA	1032463428
AREA	BOOTCAMP DE PROGRAMACIÓN WEB
FECHA	27 de septiembre de 2026
ACADEMIAS	BIT Y UNIVERSIDAD IBEROAMERICANA
CIUDAD	BOGOTÁ D.C
DOCENTE	SEBASTIAN AMAYA

Sistema de Gestión de Tareas (TODO List)

*Aplicación web full-stack con autenticación de usuarios y
gestión de tareas personales*

1. Resumen ejecutivo

Este proyecto consiste en el desarrollo de una aplicación web completa (full-stack) para la gestión personal de tareas, orientada a que cada usuario registrado pueda organizar sus pendientes de forma privada, organizada y segura.

La solución está compuesta por dos capas independientes que se comunican entre sí a través de una API REST: un backend construido en Node.js según el ejemplo visto en clase, con Express y MongoDB, encargado de la lógica de negocio, la persistencia de datos y la autenticación de usuarios; y un frontend construido en Angular, pero también adaptable con peticiones en postman, encargado de ofrecer una interfaz visual, muy sencilla, minimalista intuitiva y responsiva para que el usuario interactúe con el sistema.

El sistema permite que cualquier persona cree una cuenta con sus datos personales, inicie sesión de forma segura mediante autenticación basada en JSON Web Tokens (JWT) según la temática desarrollada en clase, y una vez autenticada, gestione un listado propio de tareas: crearlas, visualizarlas, editarlas, marcarlas como completadas y eliminarlas.

Toda la información de tareas está asociada exclusivamente al usuario que la creó, y nadie más podrá acceder a ellas a menos que sea el mismo usuario que las creó, garantizando privacidad entre distintas cuentas.

2. Objetivos del proyecto

2.1 Objetivo general

Diseñar, desarrollar e implementar una aplicación web full-stack que permita a los usuarios que la utilicen gestionar sus tareas personales de manera segura, eficaz y sencilla, aplicando buenas prácticas de desarrollo backend según como aprendimos en clase y desarrollo frontend básico con angular.

2.2 Objetivos específicos

- Implementar un sistema de registro y autenticación de usuarios mediante JSON Web Tokens (JWT), protegiendo el acceso a los recursos de la aplicación.

- Diseñar y construir una API REST en Node.js y Express que exponga operaciones CRUD (crear, leer, actualizar, eliminar) sobre las tareas de cada usuario.
- Modelar la persistencia de datos utilizando MongoDB y Mongoose, definiendo esquemas claros para usuarios y tareas.
- Aplicar validaciones de entrada en el backend para garantizar la integridad de los datos.
- Desarrollar una interfaz de usuario en Angular, organizada en componentes independientes (login, registro, listado y formulario de tareas), consumiendo la API mediante servicios HTTP.
- Proteger las rutas del frontend que requieren autenticación, evitando el acceso de usuarios no autenticados a la gestión de tareas.
- Garantizar la comunicación correcta entre frontend y backend mediante la configuración de políticas CORS que no fueron vistas en clase, pero que son indispensables a la hora de una petición o solicitud de login.
- Ofrecer retroalimentación visual clara al usuario ante cada acción realizada (creación, edición, eliminación, errores de validación).

3. Alcance del proyecto

El proyecto abarca el ciclo completo básico y elemental de una funcionalidad típica de gestión de información personal: desde el registro de un nuevo usuario, con sus datos personales, pasando por la autenticación segura, hasta la administración completa (crear, leer, actualizar y eliminar) de un recurso propio del usuario, en este caso, sus tareas.

No incluye funcionalidades adicionales como recuperación de contraseña, edición del perfil de usuario, ni colaboración entre distintos usuarios sobre una misma tarea, ya que el objetivo principal fue centrarse en demostrar el dominio de la arquitectura básica cliente-servidor, la autenticación basada en tokens y el consumo de servicios REST desde un framework frontend moderno sencillo y funcional.

4. Arquitectura de la solución

La aplicación sigue una arquitectura cliente-servidor desacoplada, donde el frontend y el backend son proyectos independientes que se comunican exclusivamente a través de peticiones HTTP sobre una API REST.

Capa	Descripción
Backend (API REST)	Node.js + Express, expone endpoints para autenticación (/api/auth) y gestión de tareas (/api/task). Se conecta a una base de datos MongoDB a través de Mongoose.
Base de datos	MongoDB, con dos colecciones principales: usuarios (con sus datos personales y contraseña hashada) y tareas (asociadas a un usuario mediante referencia).
Frontend (SPA)	Angular (standalone components), consume la API mediante HttpClient, gestiona el estado local con signals, y protege sus rutas mediante guards e interceptores.
Autenticación	JSON Web Tokens (JWT). El backend firma un token al iniciar sesión; el frontend lo almacena y lo envía en el header Authorization de cada petición protegida.

5. Funcionalidades principales

5.1 Módulo de autenticación

- Registro de nuevos usuarios, solicitando nombre, correo electrónico, contraseña, edad, sexo y fecha de nacimiento.
- Validación de la fortaleza de la contraseña (mínimo 6 caracteres, combinación de mayúsculas, minúsculas, números y símbolos).
- Inicio de sesión mediante correo y contraseña, con generación de un token JWT válido por una hora.

- Cierre de sesión, eliminando el token almacenado en el navegador.
- Redirección automática al login cuando el token es inválido, ha expirado, o el usuario intenta acceder a una ruta protegida sin estar autenticado.

5.2 Módulo de gestión de tareas

- Listado de todas las tareas pertenecientes al usuario autenticado.
- Creación de una nueva tarea, con título y descripción.
- Edición de una tarea existente (título, descripción o estado de completado).
- Marcado rápido de una tarea como completada o pendiente mediante una casilla de verificación.
- Eliminación de una tarea, con confirmación previa del usuario.

5.3 Experiencia de usuario

- Interfaz limpia y responsiva, adaptable a distintos tamaños de pantalla.
- Notificaciones visuales tipo "toast" que confirman el resultado de cada acción (creación, edición, eliminación).
- Mensajes de error claros y específicos ante fallos de validación o de comunicación con el servidor.
- Navegación protegida: solo un usuario autenticado puede acceder al listado y gestión de tareas.

6. Tecnologías utilizadas

Componente	Tecnologías
Backend	Node.js, Express, Mongoose, JSON Web Token, bcrypt, express-validator, cors, dotenv
Base de datos	MongoDB
Frontend	Angular (standalone components), TypeScript, RxJS, Reactive Forms, HttpClient
Herramientas	Angular CLI, npm, Postman/curl (pruebas de API), Git

7. Usuario objetivo y casos de uso

La aplicación está dirigida a cualquier persona, empresa o emprendimiento que necesite organizar tareas o pendientes personales de forma sencilla y organizada, sin necesidad de compartirlas con otros usuarios ni de configuraciones complejas.

Un caso de uso representativo es el de un estudiante o profesional que desea llevar el control de sus actividades diarias, marcando cuáles ya completó y cuáles siguen pendientes, desde cualquier dispositivo con acceso a un navegador web.

8. Resultados y aprendizajes obtenidos

Durante el desarrollo del proyecto se identificaron y corrigieron distintos errores propios tanto del proceso de construcción de software, como de algunos conceptuales entre ellos:

inconsistencias en nombres de variables y campos entre el modelo de datos y la lógica de negocio, ausencia de protección de autenticación en algunas rutas, y la necesidad de habilitar políticas de intercambio de recursos entre

distintos orígenes (CORS) para permitir la comunicación entre el frontend y el backend, al ejecutarse en puertos diferentes durante el desarrollo local.

La resolución de estos problemas me permitió reforzar conceptos fundamentales del desarrollo backend, como el ciclo de vida de una petición HTTP, el funcionamiento de los middlewares en Express, el alcance de las variables en JavaScript, y la importancia de mantener consistencia entre los esquemas de base de datos y el código que los manipula.

Del lado del frontend, se reforzaron conceptos de arquitectura de aplicaciones Angular modernas, como el uso de componentes independientes (standalone), inyección de dependencias, guards de ruta, interceptores HTTP y manejo de estado reactivo con signals.

9. Conclusiones

El proyecto cumple con los objetivos planteados: se logró construir una aplicación funcional de principio a fin sencilla, con una arquitectura clara y separación de responsabilidades entre el backend y el frontend, un sistema de autenticación seguro basado en JWT, y una interfaz de usuario funcional que permite gestionar tareas personales de manera completa, secuencial e intuitiva.

El resultado es una base sólida que demuestra la aplicación práctica de conceptos de desarrollo web full-stack, y que puede ser extendida en el futuro con funcionalidades adicionales como recuperación de contraseña, edición de perfil, o colaboración entre usuarios.

10. términos de evaluación

- Enlace del repositorio en github con trabajo en ramas y commits

<https://github.com/Desingsoft/Actividad-2-de-Node-Js>